DEEP LEARNING

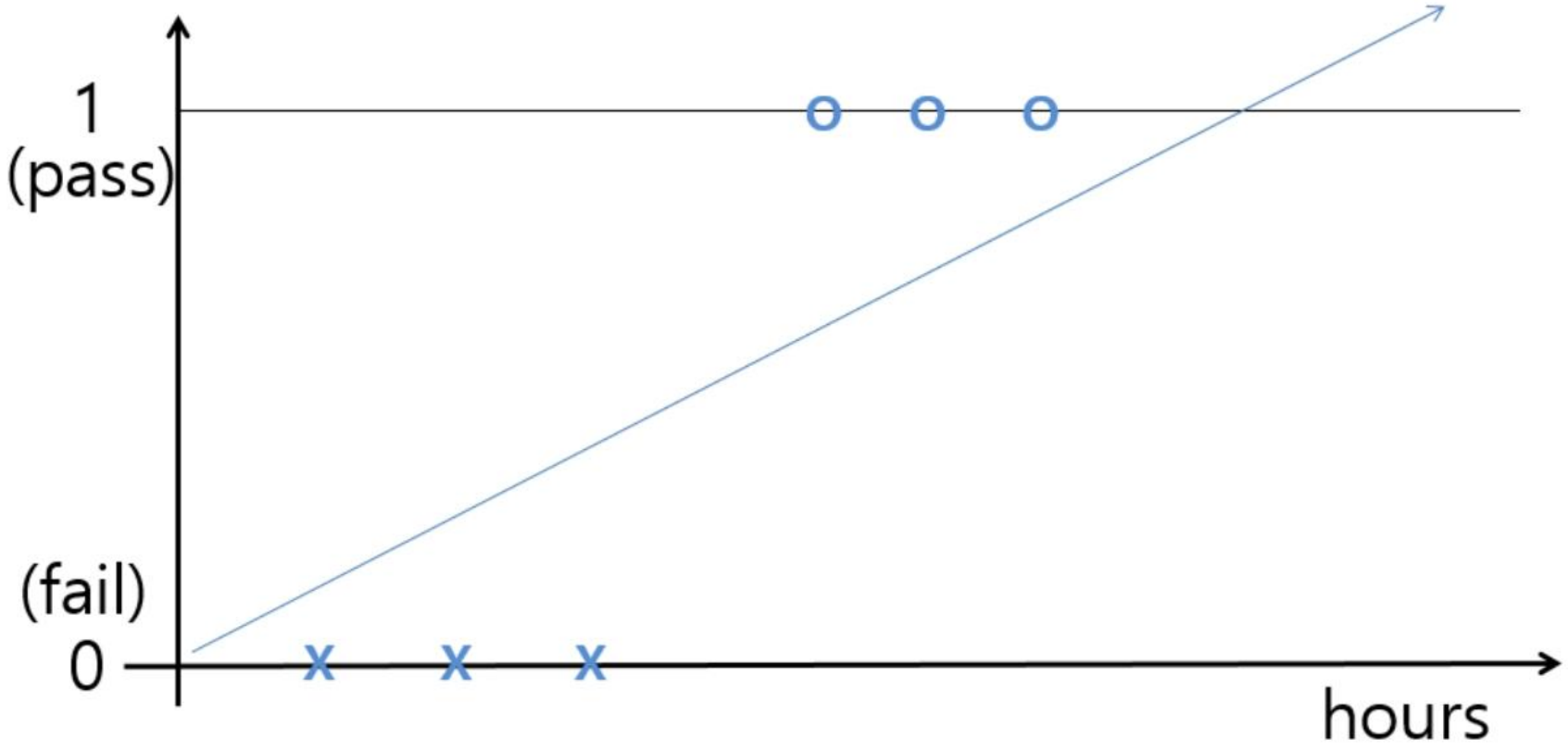
LOGISTIC CLASSIFICATION

CLASSIFICATION AND ENCODING

- Spam Detection: Spam or Ham
- Facebook feed: show or hide
- Credit Card Fraudulent Transaction detection: legitimate/fraud

- Spam Detection: Spam (1) or Ham (0)
- Facebook feed: show(1) or hide(0)
- Credit Card Fraudulent Transaction detection: legitimate(0) or fraud (1)

LINEAR REGRESSION?

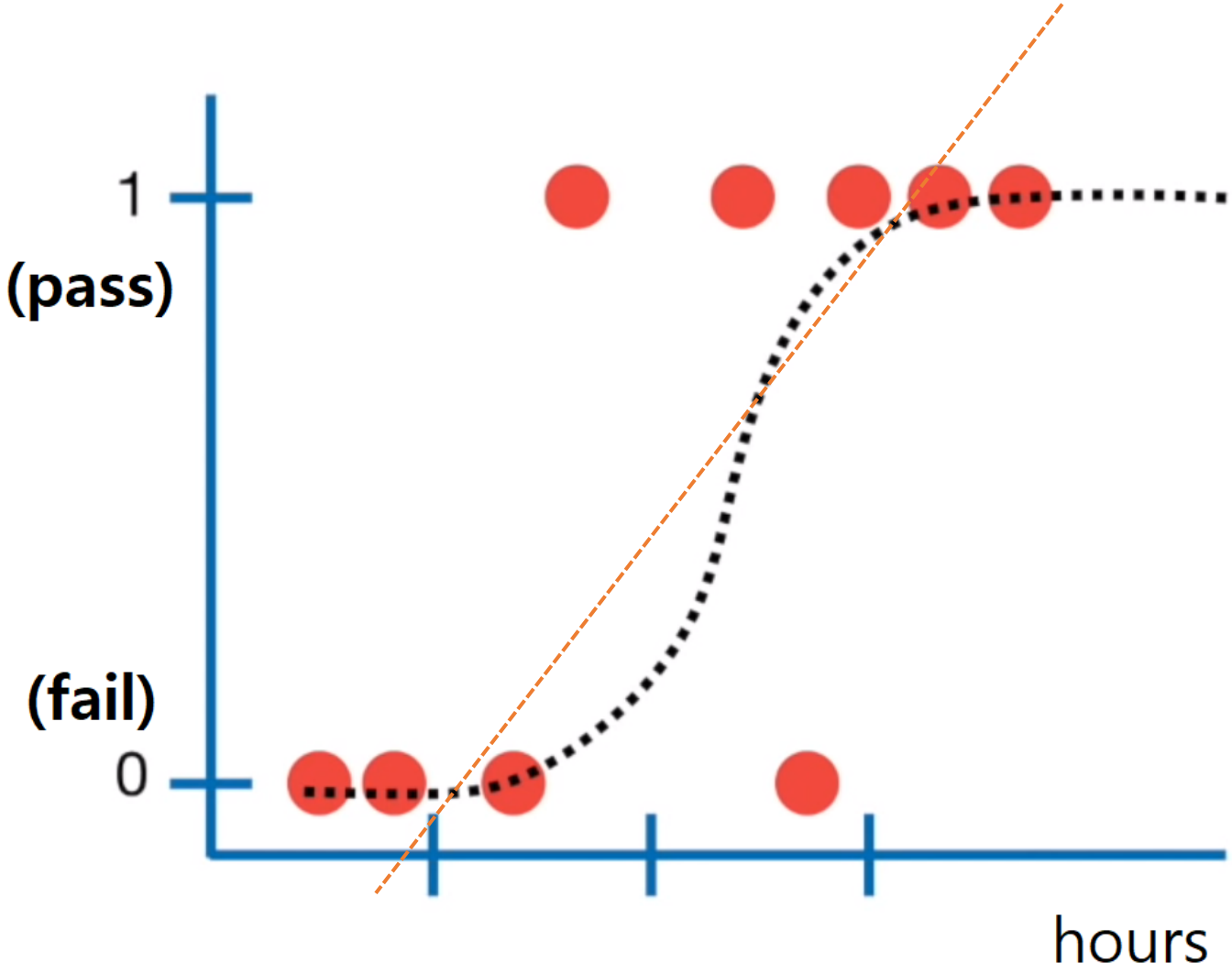


PROBLEMS

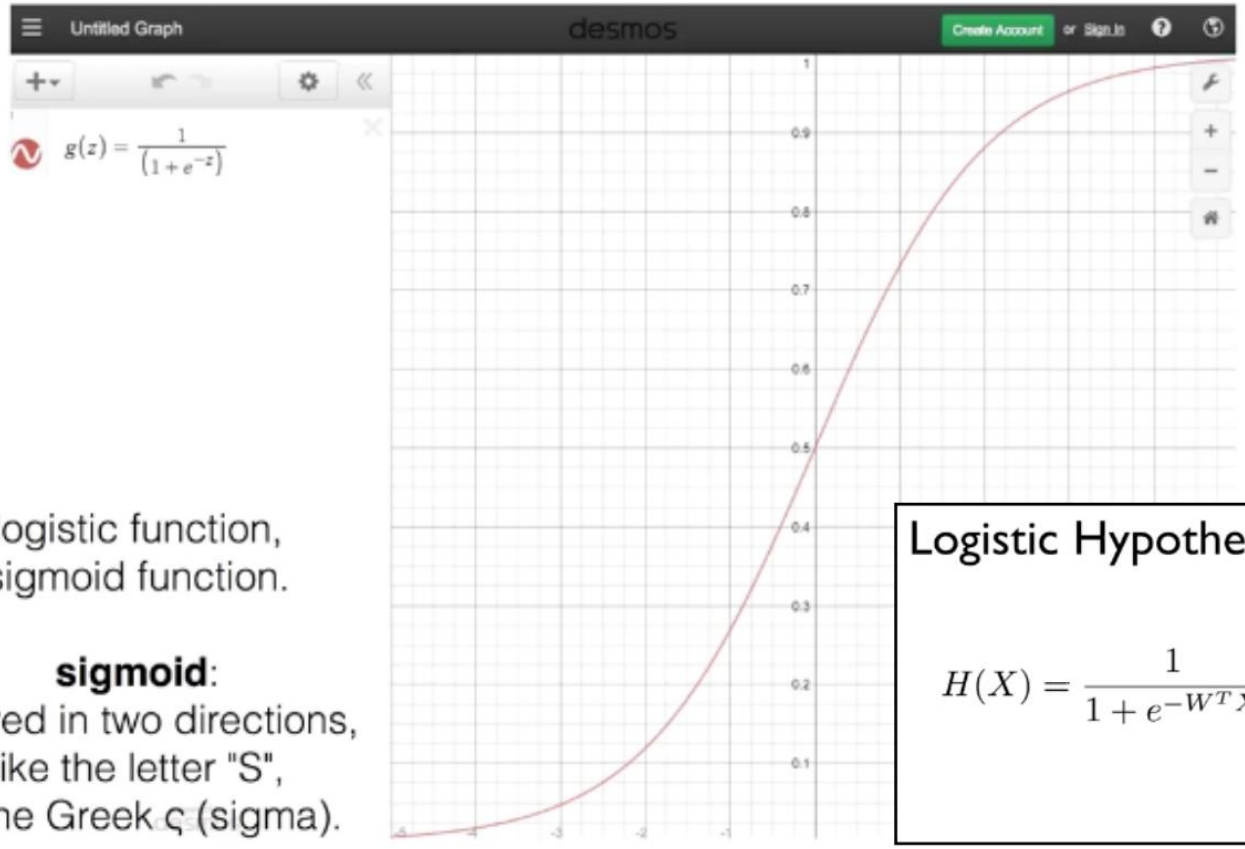
Linear regression

- We know Y is 0 or 1
  $H(x) = Wx + b$
- Hypothesis can give values large than 1 or less than 0

LINEAR REGRESSION?



SIGMOID



logistic function,
sigmoid function.

sigmoid:
Curved in two directions,
like the letter "S",
or the Greek ς (sigma).

Logistic Hypothesis

$$H(X) = \frac{1}{1+e^{-W^T X}}$$

COST FUNCTION

$$cost(W, b) = \frac{1}{m} \sum_{i=1}^{m} (H(x^{(i)}) - y^{(i)})^2$$

$$H(x) = Wx + b$$

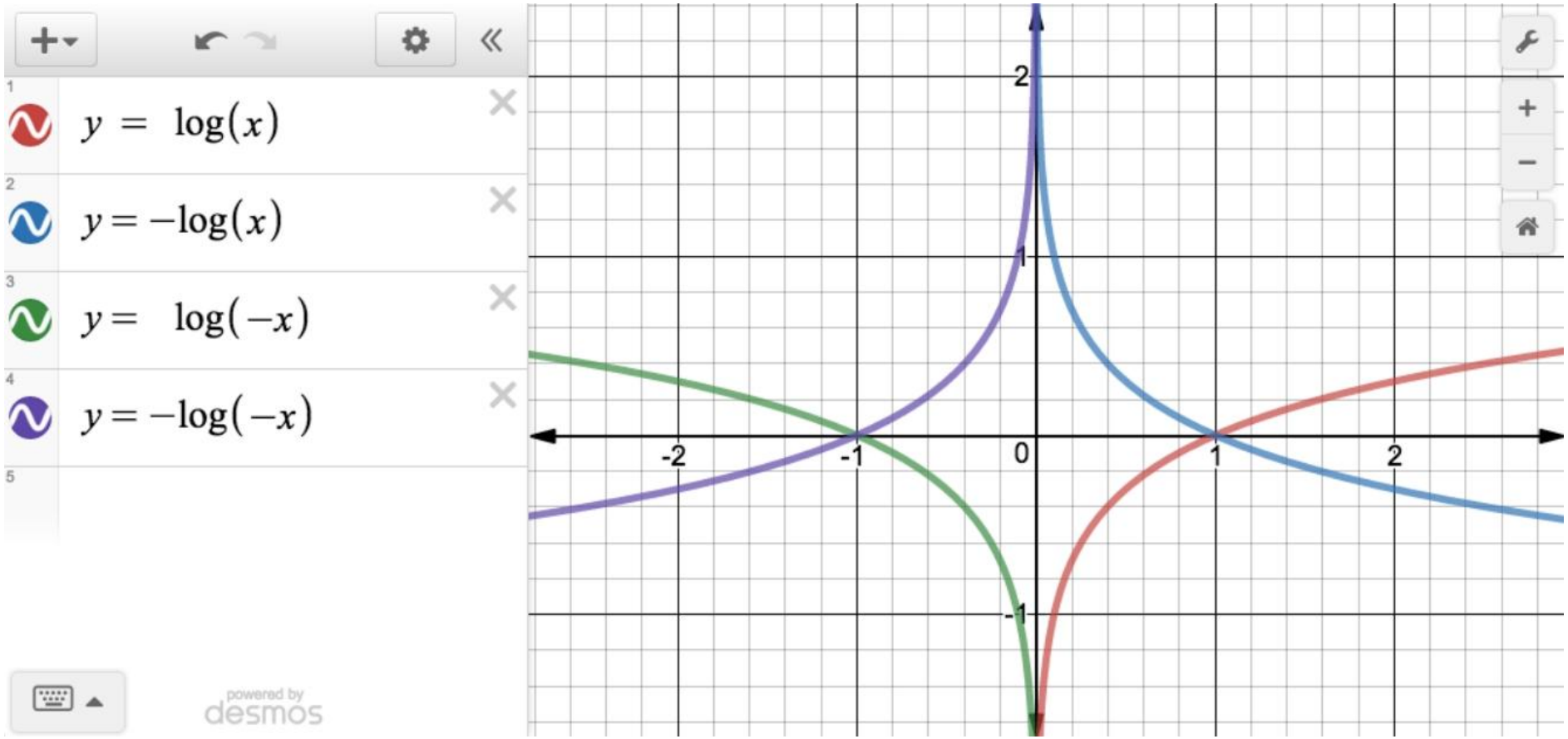
$$H(X) = \frac{1}{1 + e^{-W^T X}}$$

NEW COST FUNCTION FOR LOGISTIC

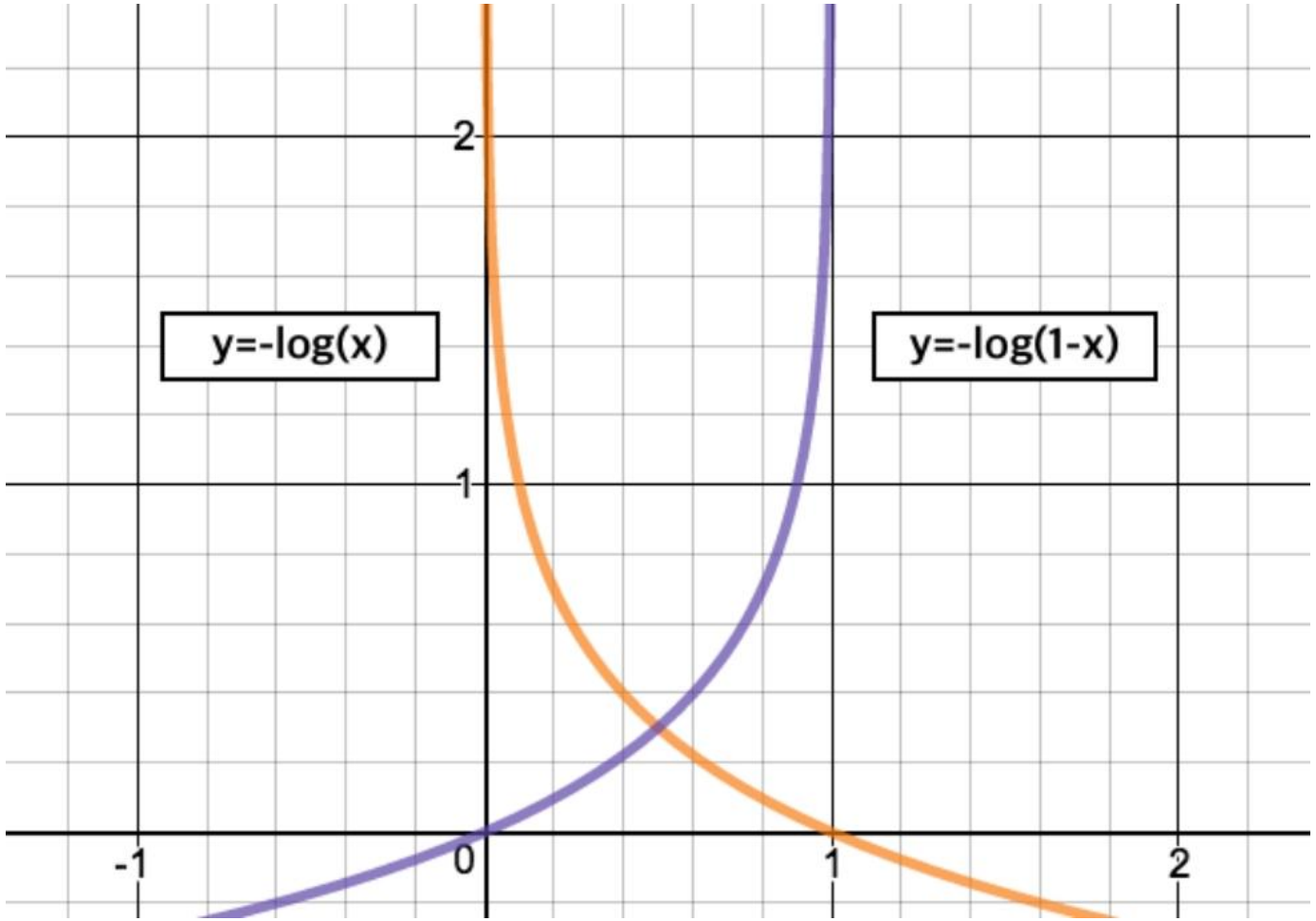
$$cost(W) = \frac{1}{m} \sum \quad c(H(x), y)$$

$$c(H(x), y) = \begin{cases} -log(H(x)) & : y = 1 \\ -log(1 - H(x)) & : y = 0 \end{cases}$$

LOG FUNCTION



LOG FUNCTIONS WE NEED TO KNOW



UNDERSTANDING COST FUNCTION

$$C(H(x),y) = \begin{cases} -\log(H(x)) & : y = 1 \\ -\log(1-H(x)) & : y = 0 \end{cases}$$

$y = 1$

$H(x) = 1$, $\text{cost}(1) = 0$

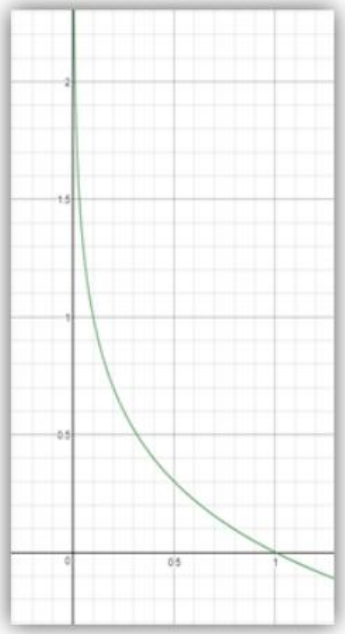
$H(x) = 0$, $\text{cost}(0) = \infty$

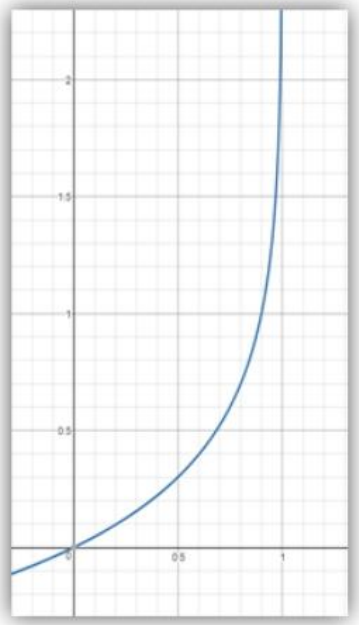
$y = 0$

$H(x) = 0$, $\text{cost}(0) = 0$

$H(x) = 1$, $\text{cost}(1) = \infty$

$g(z) = -\log(z)$





$g(z) = -\log(1-z)$

COST FUNCTION

$$cost(W) = \frac{1}{m} \sum \quad c(H(x), y)$$

$$c(H(x), y) = \begin{cases} -log(H(x)) & : y = 1 \\ -log(1 - H(x)) & : y = 0 \end{cases}$$

$$c(H(x), y) = -ylog(H(x)) - (1 - y)log(1 - H(x))$$

GRADIENT DESCENT ALGORITHM

$$cost(W) = -\frac{1}{m}\sum ylog(H(x)) + (1-y)log(1-H(x))$$

$$W := W - \alpha\frac{\partial}{\partial W}cost(W)$$

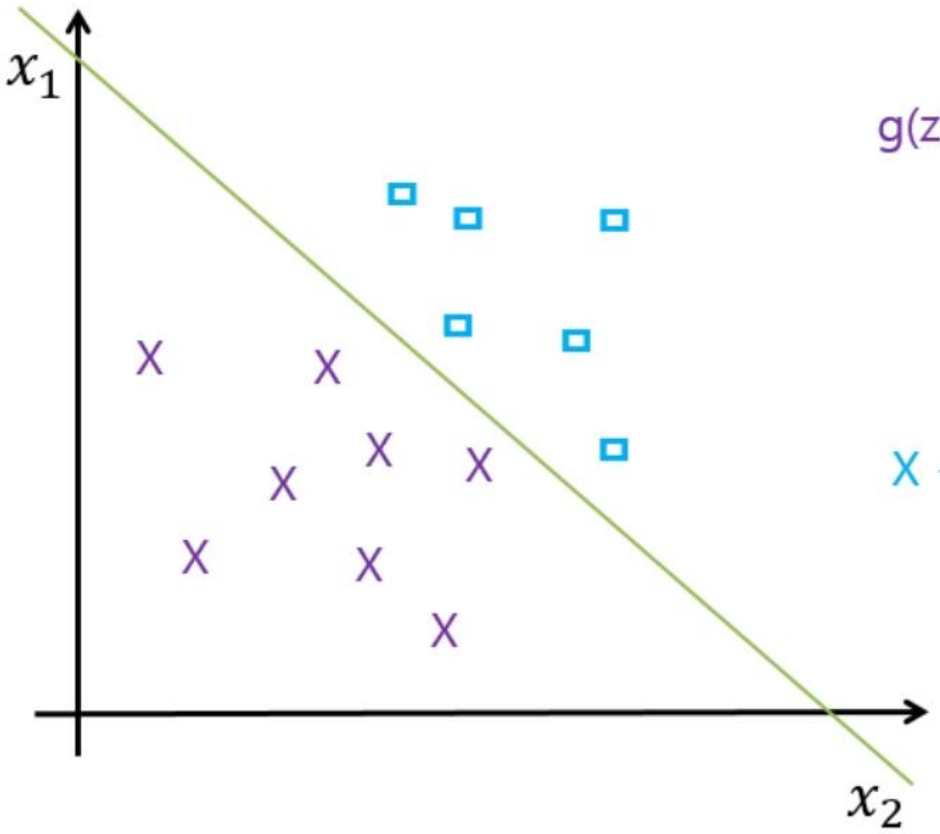
```
# cost function
cost =-tf.reduce_mean(Y*tf.log(hypothesis) + (1-Y)*tf.log(1-hypothesis))

# Minimize
a = tf.Variable(0.1) # Learning rate, alpha
optimizer = tf.train.GradientDescentOptimizer(a)
train = optimizer.minimize(cost)
```

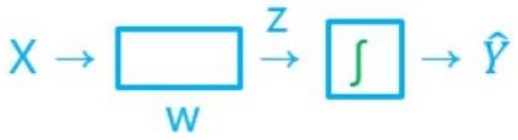
DEEP LEARNING

SOFTMAX CROSS-ENTROPY

LOGISTIC REGRESSION

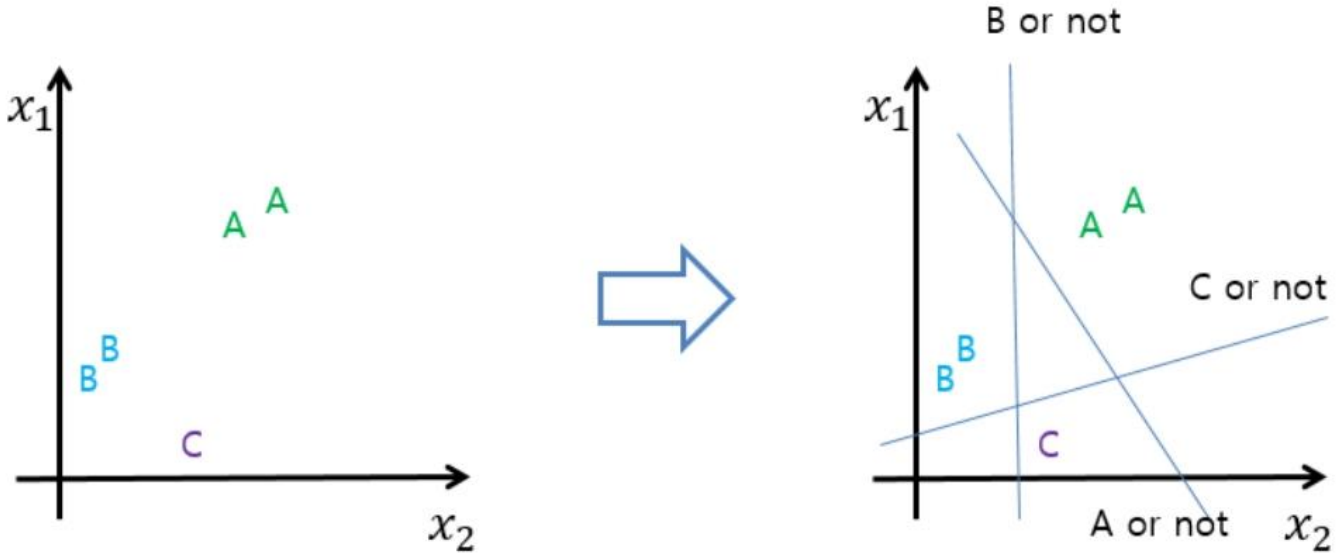


$$g(z) = \frac{1}{1+e^{-z}}$$

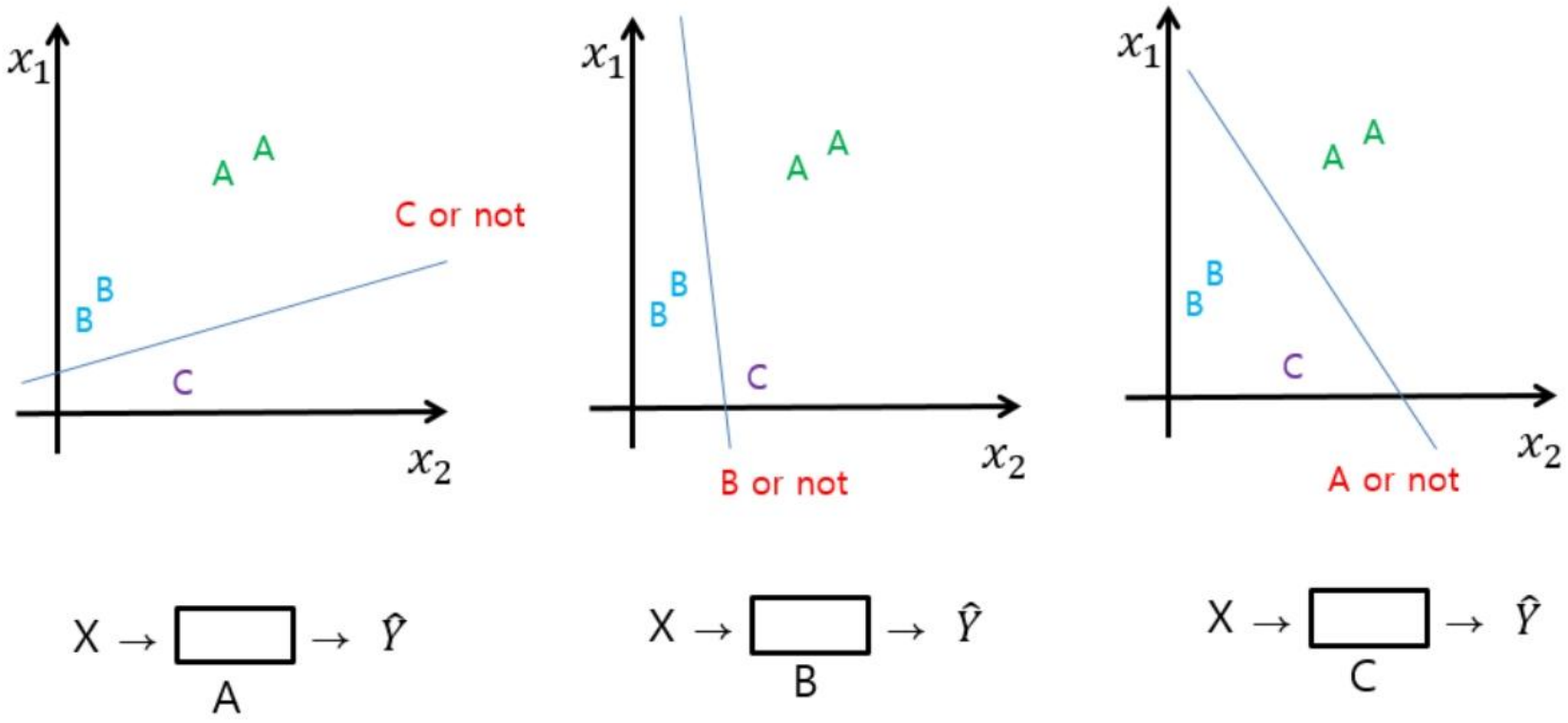


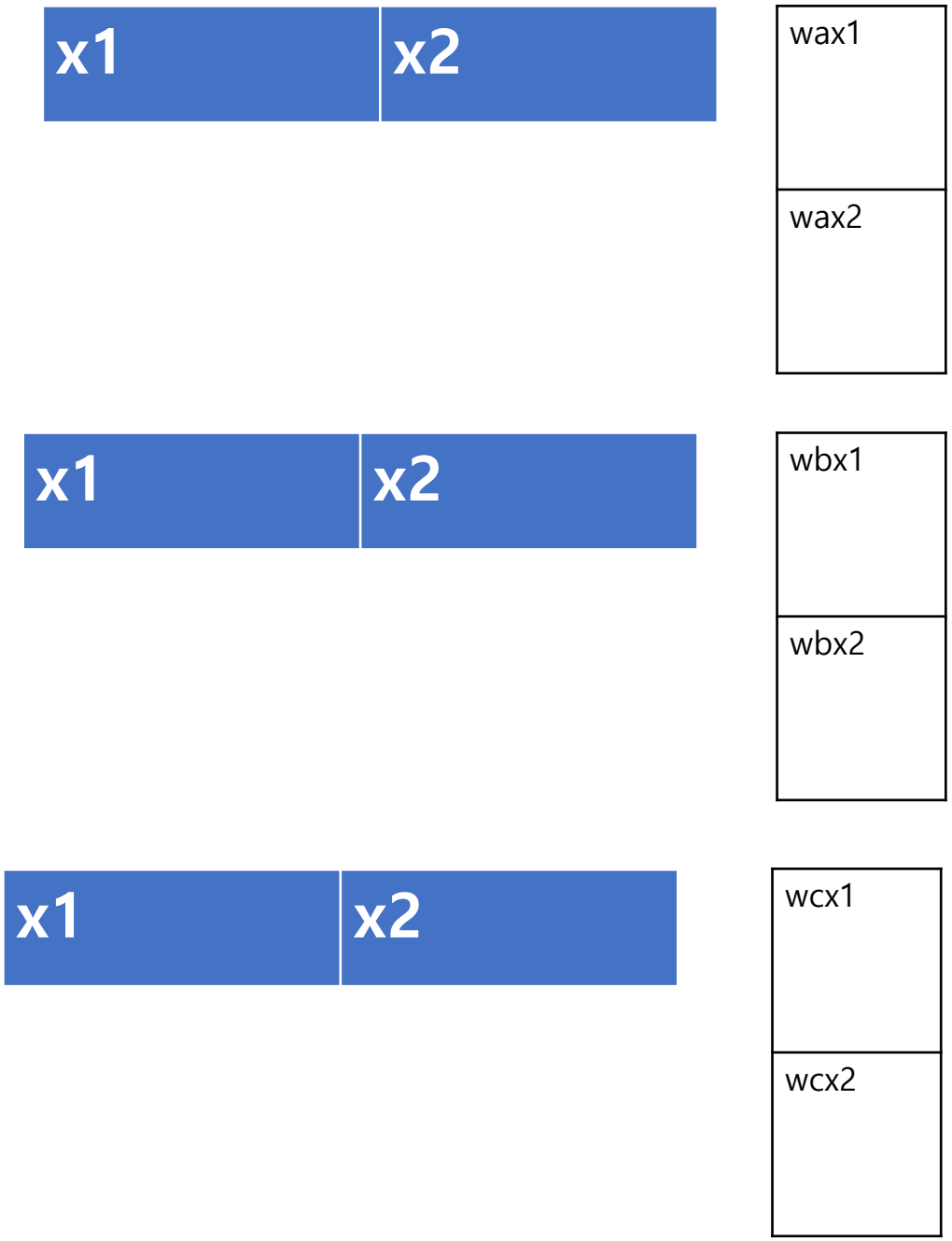
MULTINOMIAL CLASSIFICATION

x1 (hours)	x2 (attendance)	y (grade)
10	5	A
9	5	A
3	2	B
2	4	B
11	1	C



MULTINOMIAL CLASSIFICATION




x1
x2
wax1
wax2
x1
x2
wbx1
wbx2
x1
x2
wcx1
wcx2

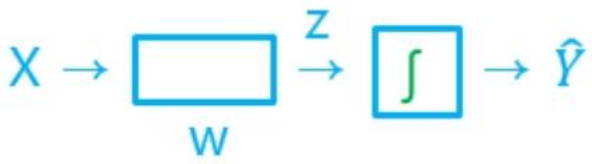
x1	x2

wax1	wbx1	wcx1
wax2	wbx2	wcx2

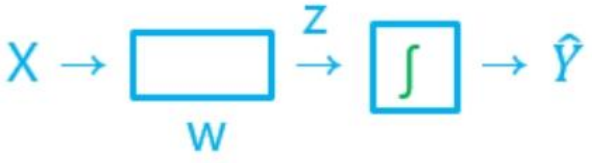
X1xWax1 + X2xWax2 +Ba
X1xWbx1 + X2xWbx2 +Bb
X1xWcx1 + X2xWcx2 +Bc

MULTINOMIAL CLASSIFICATION

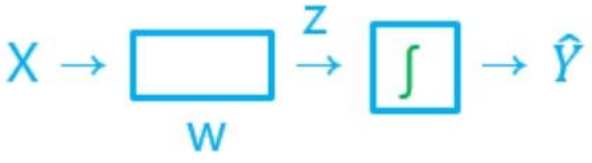
$$[w_1 \quad w_2 \quad w_3]\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [w_1x_1 + w_2x_2 + w_3x_3]$$



$$[w_1 \quad w_2 \quad w_3]\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [w_1x_1 + w_2x_2 + w_3x_3]$$



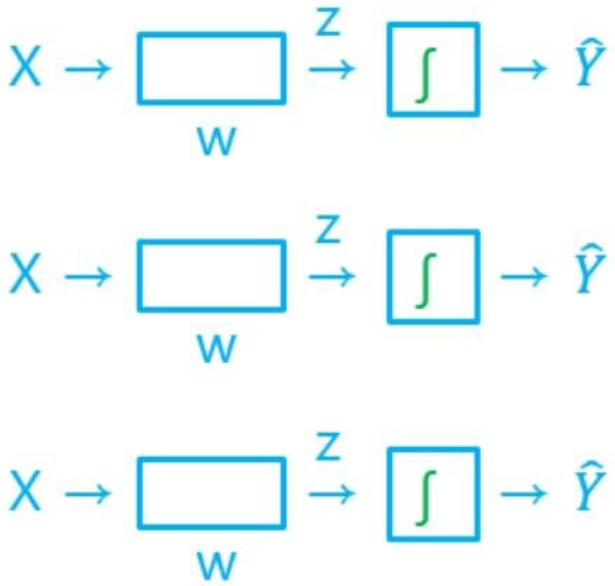
$$[w_1 \quad w_2 \quad w_3]\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [w_1x_1 + w_2x_2 + w_3x_3]$$



MULTINOMIAL CLASSIFICATION

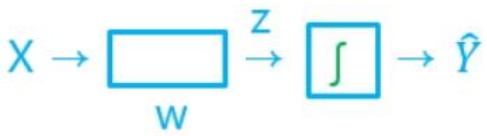
$$[w_1 \;\; w_2 \;\; w_3]\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [w_1x_1 + w_2x_2 + w_3x_3]$$

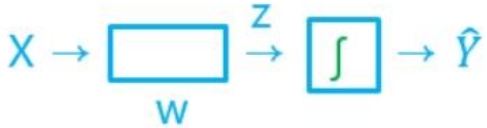
$$\begin{bmatrix} w_{A1} & w_{A2} & w_{A3} \\ w_{B1} & w_{B2} & w_{B3} \\ w_{C1} & w_{C2} & w_{C3} \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \;?$$

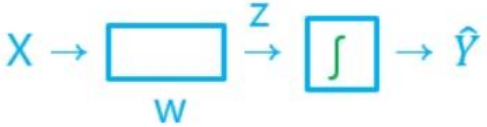


MULTINOMIAL CLASSIFICATION

$$\begin{bmatrix} w_{A1} & w_{A2} & w_{A3} \\ w_{B1} & w_{B2} & w_{B3} \\ w_{C1} & w_{C2} & w_{C3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} w_{A1}x_1 + w_{A2}x_2 + w_{A3}x_3 \\ w_{B1}x_1 + w_{B2}x_2 + w_{B3}x_3 \\ w_{C1}x_1 + w_{C2}x_2 + w_{C3}x_3 \end{bmatrix} = \begin{bmatrix} \hat{Y}_A \\ \hat{Y}_B \\ \hat{Y}_C \end{bmatrix}$$



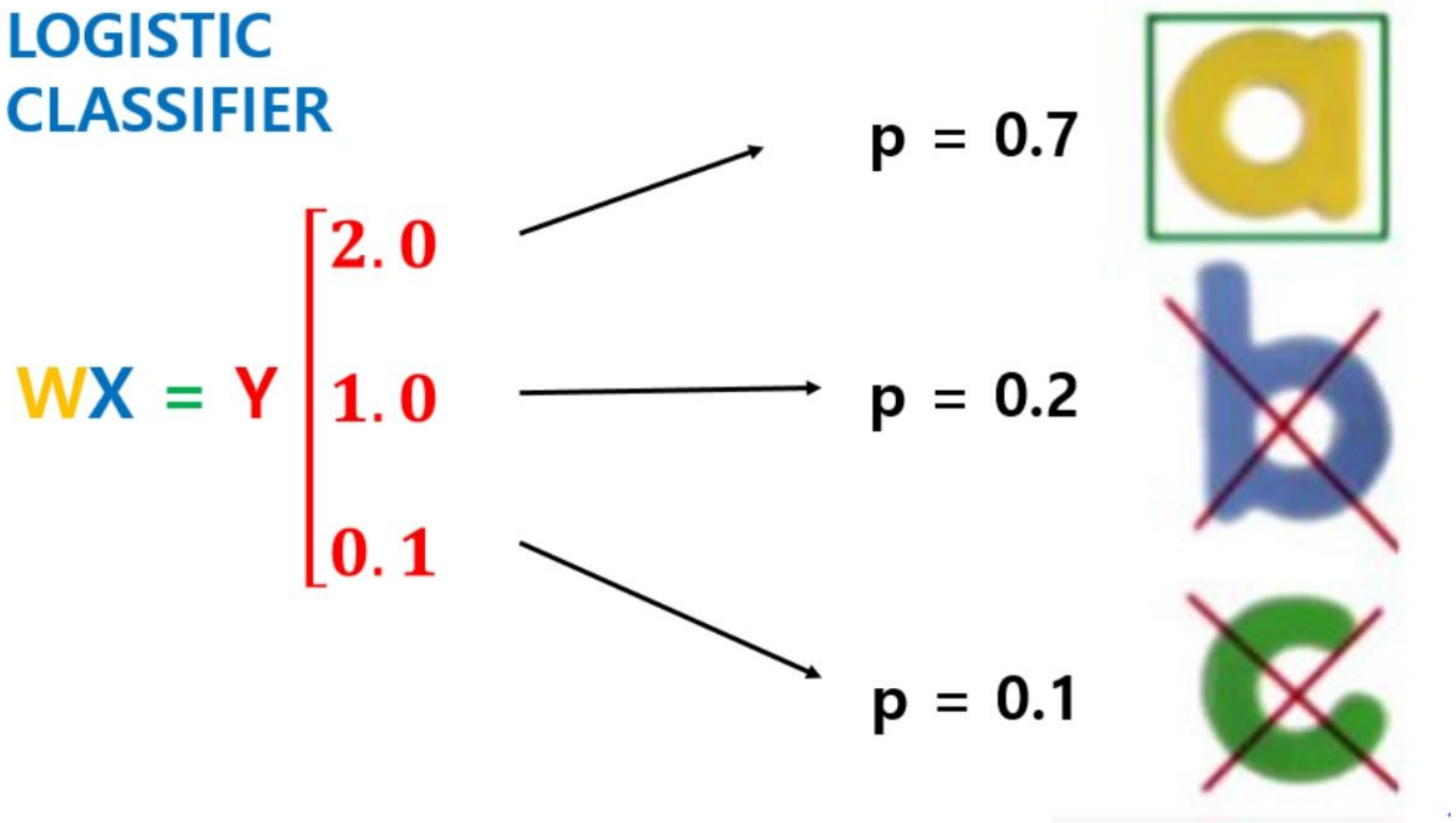




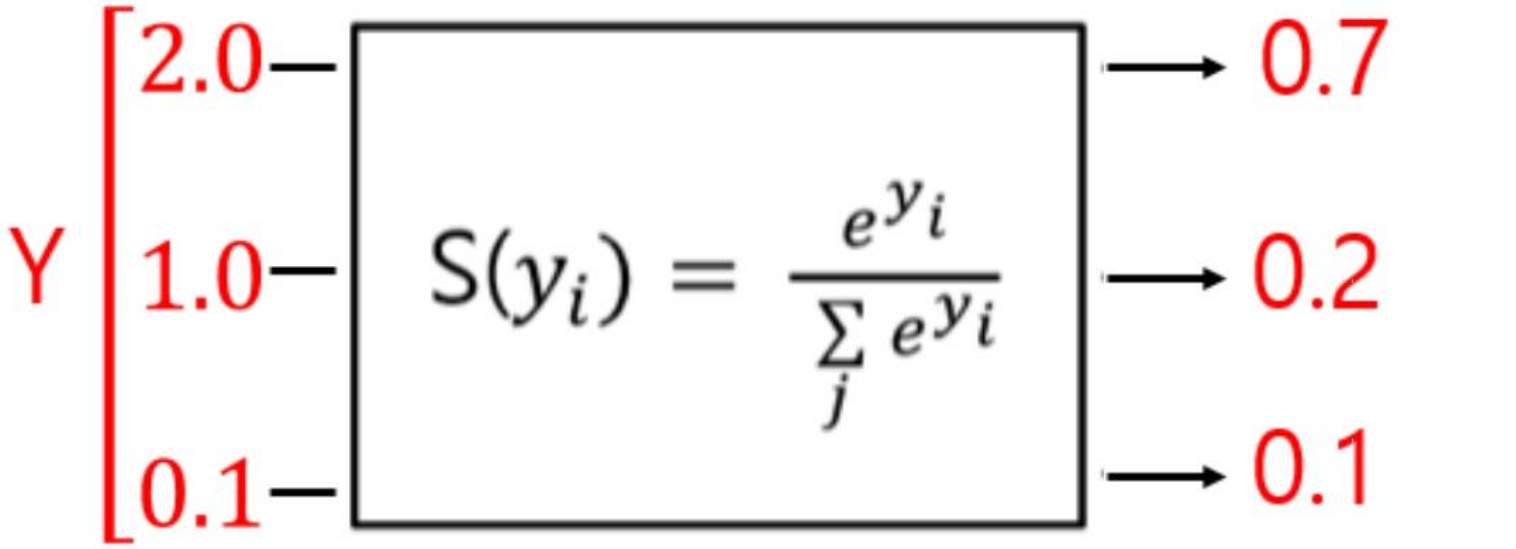
WHERE IS SIGMOID?

$$\begin{bmatrix} w_{A1} & w_{A2} & w_{A3} \\ w_{B1} & w_{B2} & w_{B3} \\ w_{C1} & w_{C2} & w_{C3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} w_{A1}x_1 + w_{A2}x_2 + w_{A3}x_3 \\ w_{B1}x_1 + w_{B2}x_2 + w_{B3}x_3 \\ w_{C1}x_1 + w_{C2}x_2 + w_{C3}x_3 \end{bmatrix} = \begin{bmatrix} \hat{Y}_A \\ \hat{Y}_B \\ \hat{Y}_C \end{bmatrix} \begin{bmatrix} \mathbf{2.0} \\ \mathbf{1.0} \\ \mathbf{0.1} \end{bmatrix}$$

SIGMOID?



SOFTMAX



SCORES → PROBABILITIES

SOFTMAX

hypothesis = tf.nn.**softmax**(tf.matmul(X,W)+b)

XW = Y

tf.matmul(X,W)+b

$$\begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix} \longrightarrow \boxed{S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_i}}} \longrightarrow \begin{matrix} 0.7 \\ 0.2 \\ 0.1 \end{matrix}$$

SCORES ⟶ PROBABILITIES

SOFTMAX_CROSS_ENTROPY_WITH_LOGITS

```
logits = tf.matmul(X, W) + b
hypothesis = tf.nn.softmax(logits)
```

1

```
# Cross entropy cost/loss
cost = tf.reduce_mean(-tf.reduce_sum(Y * tf.log(hypothesis), axis=1))
```

2

```
# Cross entropy cost/loss
cost_i = tf.nn.softmax_cross_entropy_with_logits(logits=logits,
                                                 labels=Y_one_hot)
cost = tf.reduce_mean(cost_i)
```

DEEP LEARNING

NEURAL NETWORKS HISTORY

a	b	and	or	xor
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

XOR PROBLEM

(Simple) XOR problem: linearly separable?

$$W = \begin{pmatrix} -11 \\ -11 \end{pmatrix}$$

$$B = 6$$

x1	x2	y1	y2	y3		
0	0	0	1	0		
0	1	0	0	1		
1	0	0	0	1		
1	1	1	0	0		

$$W = \begin{pmatrix} 5 \\ 5 \end{pmatrix} \qquad W = \begin{pmatrix} -7 \\ -7 \end{pmatrix}$$

$$B = -8 \qquad B = 3$$

$$\begin{pmatrix} 1 & 0 \end{pmatrix} \times \begin{pmatrix} -11 \\ -11 \end{pmatrix} + 6$$

$$= \begin{pmatrix} \\ \end{pmatrix}$$

$$W = \begin{pmatrix} -11 \\ -11 \end{pmatrix}$$

$$B = 6$$

x1	x2	y1	y2	y3		
0	0	0	1	0		
0	1	0	0	1		
1	0	0	0	1		
1	1	1	0	0		

$$\begin{pmatrix} 0 & 0 \end{pmatrix} \times \begin{pmatrix} -11 \\ -11 \end{pmatrix} + 6$$

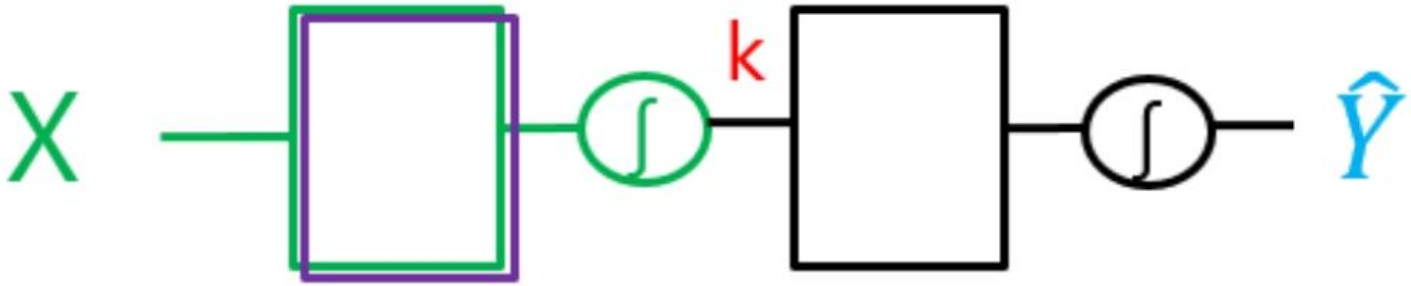
$$= \begin{pmatrix} \\ \\ \end{pmatrix}$$

$$W = \begin{pmatrix} 5 \\ 5 \end{pmatrix} \begin{pmatrix} -7 \\ -7 \end{pmatrix}$$

$$B = -8 \qquad B = 3$$

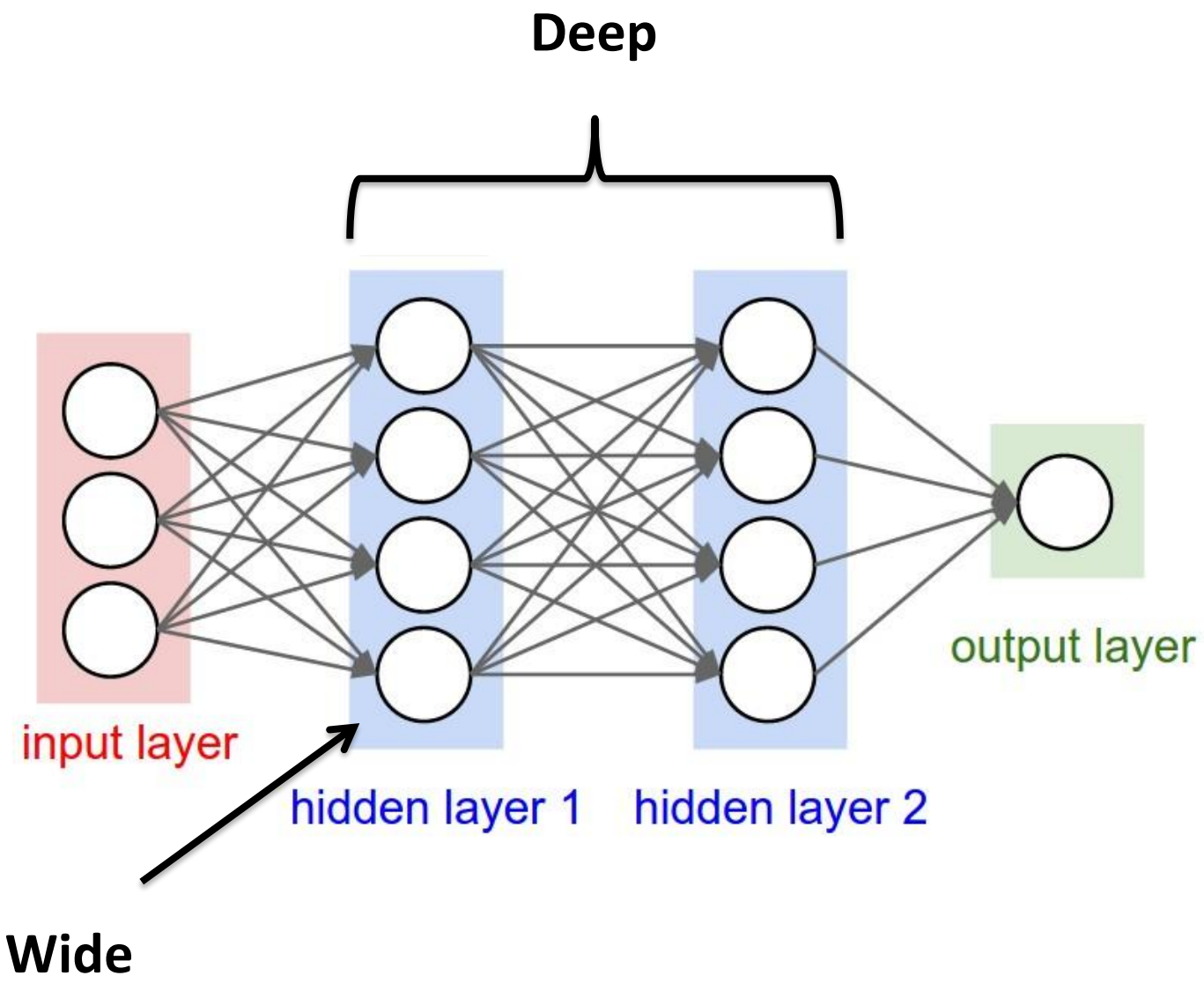
NEURAL NETWORK FOR XOR

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

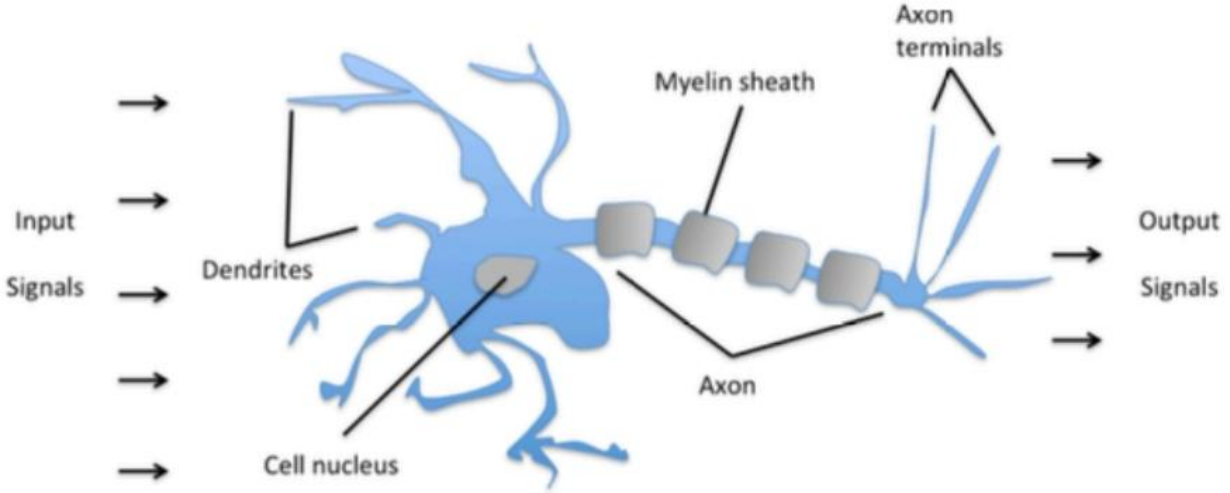


$$W_1 = \begin{bmatrix} 5 & -7 \\ 5 & -7 \end{bmatrix} \qquad W_2 = \begin{bmatrix} -11 \\ -11 \end{bmatrix}$$

$$B_1 = \begin{bmatrix} -8 & 3 \end{bmatrix} \qquad B_2 = 6$$


Deep
input layer
hidden layer 1
hidden layer 2
output layer
Wide

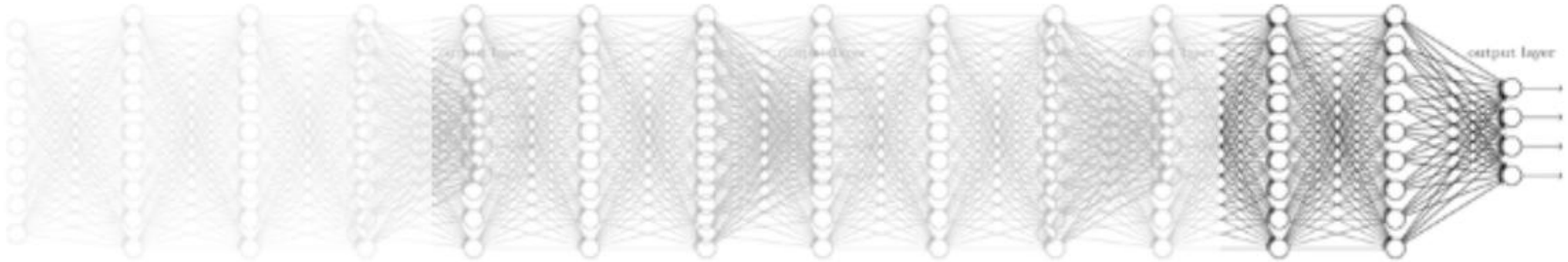
SCHEMATIC OF A BIOLOGICAL NEURON



Schematic of a biological neuron.

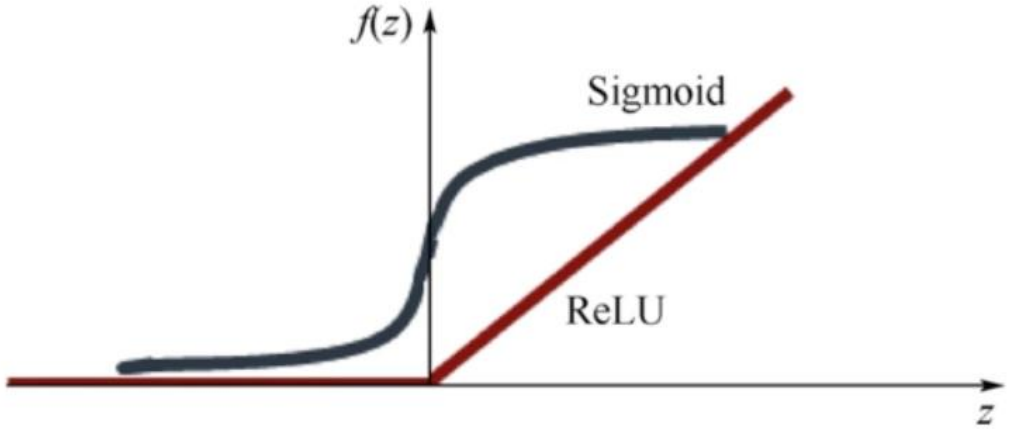
VANISHING GRADIENT

Vanishing gradient (NN winter2: 1986-2006)



SIGMOID VS. RELU

Sigmoid!



ReLU: Rectified Linear Unit

```
L1 = tf.sigmoid(tf.matmul(X, W1) + b1)

L1 = tf.nn.relu(tf.matmul(X, W1) + b1)
```